

OOP principles in my project

Inheritance: Shared functionality is defined once in one class and reused by all subclasses.

```
public class Player extends Entity {  
public class GamePanel extends JPanel {  
public class GameWindow extends JFrame {  
public class KeyboardInputs implements KeyListener
```

Encapsulation: Controlled access to internal state prevents misuse

```
private BufferedImage[][] animations; 5 usages  
private int aniTick, aniIndex, aniSpeed = 25; 4 usages  
private int playerAction = IDLE; 8 usages  
private boolean moving = false, attacking = false; 3  
private boolean left, up, right, down; 6 usages  
private float playerSpeed = 2.0f; 2 usages  
private int[][] lvlData; 4 usages  
private float xDrawOffset = 38 * Game.SCALE; 1 usage  
private float yDrawOffset = 4 * Game.SCALE; 1 usage
```

Most of the class fields are marked as private, which means they cannot be accessed directly from outside the Player class. This is a key aspect of encapsulation, as it ensures that the internal state of the object is protected and cannot be modified directly by external classes (need getter and setter)

```

public void setAttacking(boolean attacking) { this.attacking = attacking; }

public boolean isAttacking() { return attacking; }

public int getPlayerAction() { return playerAction; }

public boolean isLeft() { return left; }

public void setLeft(boolean left) { this.left = left; }

public boolean isUp() { return up; }

public void setUp(boolean up) { this.up = up; }

public boolean isRight() { return right; }

public void setRight(boolean right) { this.right = right; }

public boolean isDown() { return down; }

public void setDown(boolean down) { this.down = down; }

```

Encapsulation involves controlling access to the private fields using public getter and setter methods, they help to access private fields

```

private void resetAniTick() { 1 usage VitaliiDiurd
private void updatePos() { 1 usage VitaliiDiurd
private void setAnimation() {

```

Many methods are private, which prevents external classes from directly calling them

```

public void loadLvldata(int[][] lvlData)

```

There are public methods that allow interaction with the Player object from outside

Abstraction

```

public abstract class Entity {

```

Entities share core functionality (position, hitbox, rendering), but their behaviors differ

Polymorphism

Allows methods to behave differently based on the object that invokes them

```
@Override  VitaliiDiurd  
public void keyPressed(KeyEvent e) {
```

Composition

Has a composition relationship with GamePanel

```
private GamePanel gamePanel;  
  
public KeyboardInputs(GamePanel gamePanel) {  
    this.gamePanel = gamePanel;  
}
```

Single Responsibility Principle(read on some source about it)

Class have one reason to change. The Single Responsibility Principle is followed in the KeyboardInputs class because it is solely responsible for handling keyboard inputs and updating the Player's movement or jump states

```
@Override  VitaliiDiurd  
public void keyPressed(KeyEvent e) {  
    switch (e.getKeyCode()) {  
        case KeyEvent.VK_W:  
            gamePanel.getGame().getPlayer().setUp(true);  
            break;  
        case KeyEvent.VK_A:  
            gamePanel.getGame().getPlayer().setLeft(true);  
            break;  
        case KeyEvent.VK_S:  
            gamePanel.getGame().getPlayer().setDown(true);  
            break;  
        case KeyEvent.VK_D:  
            gamePanel.getGame().getPlayer().setRight(true);  
            break;  
        case KeyEvent.VK_SPACE:  
            gamePanel.getGame().getPlayer().setJump(true);  
            break;  
    }  
}
```

Loose Coupling

Loose coupling means that classes are designed to be independent from each other, reducing dependencies between components

```
gamePanel.getGame().getPlayer().setUp(false);
```

Observer Pattern

Used in GUI systems to handle user input

```
@Override  VitaliiDiurd  
public void keyReleased(KeyEvent e) {  
    switch (e.getKeyCode()) {
```

Constants

Readability of the code

```
public static final int IDLE = 0; 4  
public static final int RUNNING = 1;  
public static final int JUMP = 2; 2  
public static final int FALLING = 3;  
public static final int GROUND = 4;  
public static final int HIT = 5; 2 u  
public static final int ATTACK = 6;
```

Data Encapsulation and Protection

By making constants final, you prevent other parts of the program from accidentally modifying important values

```
final int JUMP = 2; 2  
final int FALLING = 3;
```